

---

---

# Clean code. Основы.

— SOLID. DRY. KISS. YAGNI. —  
Паттерны проектирования.

---

---

# SOLID. SRP

Single Responsibility Principle — принцип единственной ответственности

Модуль должен иметь только одну причину для изменения. Или: модуль должен отвечать только за одну заинтересованную группу.

Таким образом если одна из групп пользователей запросила изменения в работу кого-то модуля, то эти изменения не должны влиять на результаты работы других групп или работы других функций.

Кроме того это как правило это приводит к тому что у каждого класса может быть не более 1 – 2 метода.

Очень важно: данный принцип также подразумевает что вся бизнес логика должна быть собрана со всего проекта в одном месте

# Single Responsibility Principle – принцип единственной ответственности

```
public class AwesomeArrayList {  
    // list logic  
  
    public AwesomeArrayList sort(Comparator<?> comparator) {  
        // sorting logic  
    }  
}
```

В JDK это вынесено в Collections

Collections.[sort](#)([List](#)<T> list)

Collections.[sort](#)([List](#)<T> list, [Comparator](#)<? super T> c)

# Open-Closed Principle – принцип открытости/закрытости

Программные сущности должны быть открыты для расширения и закрыты для изменения.

Когда вы меняете текущее поведение класса, эти изменения сказываются на всех системах, работающих с данным классом. Если хотите, чтобы класс выполнял больше операций, то идеальный вариант – не заменять старые на новые, а добавлять новые к уже существующим.

Принцип служит для того, чтобы делать поведение класса более разнообразным, не вмешиваясь в текущие операции, которые он выполняет. Благодаря этому вы избегаете ошибок в тех фрагментах кода, где задействован этот класс.

# Open-Closed Principle – принцип открытости/закрытости

```
interface List {  
    void add(String element);  
}
```

*/\* добавили требование*

*\* Замерять время выполнения всех операций нашего листа*

*\* Пришлось вносить изменения в текущую реализацию и плодить новые*

*баги*

*\* \*/*

```
public class AwesomeArrayList implements List {  
    @Override  
    public void add(String element) {  
        long timeStart = System.currentTimeMillis();  
        // adding logic  
        long timeEnd = System.currentTimeMillis() - timeStart;  
    }  
}
```

# Liskov Substitution Principle – принцип подстановки Барбары Лисков

Имеет сложное математическое определение, которое можно заменить на: Функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа, не зная об этом.

Это видно на простом примере создания объекта Collection.

```
List <String> strings = new ArrayList<>();
```

Также очень важно! Реализация в классах наследниках не должна противоречить реализации в базовых классах. В том числе нельзя закрывать методы в дочерних классах.

Нельзя вызывать методы не характерные для базовой реализации

# Liskov Substitution Principle — принцип подстановки Барбары Лисков

```
class NotImplementedException extends RuntimeException {}
```

```
public class AwesomeArrayList implements java.util.List<?> {
```

```
    @Override
```

```
    public int size() {
```

```
        throw new NotImplementedException();
```

```
    }
```

```
    @Override
```

```
    public boolean isEmpty() {
```

```
        throw new NotImplementedException();
```

```
    }
```

```
    @Override
```

```
    public boolean contains(Object o) {
```

```
        throw new NotImplementedException();
```

```
    }
```

```
    // implemented logic
```

```
}
```

## Interface Segregation Principle – принцип разделения интерфейсов

Make fine grained interfaces that are client specific.

Под интерфейсом здесь понимается именно Java интерфейс. Разделение интерфейса облегчает использование и тестирование модулей.

Много интерфейсов лучше чем один и с кучей функциональности.

# Dependency Inversion Principle – принцип инверсии зависимости

Depend on abstractions, not on concretions.

Модули верхних уровней не должны зависеть от модулей нижних уровней. Оба типа модулей должны зависеть от абстракций.

Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

Что такое модули верхних уровней? Как определить этот уровень? Как оказалось, все очень просто. Чем ближе модуль к вводу/выводу, тем ниже уровень модуля. Т.е. модули, работающие с BD, интерфейсом пользователя, низкого уровня. А модули, реализующие бизнес-логику — высокого уровня.

# Dependency Inversion Principle – принцип инверсии зависимости

```
interface List {  
}  
  
class QuickSort {  
    void sort(AwesomeArrayList list) {}  
  
    // придётся добавить еще один метод для новой реализации.  
  
    void sort(MoreAwesomeArrayList list) {}  
    // Мы завязались на реализации, а не на абстракцию (интерфейс).  
}  
  
class AwesomeArrayList implements List {  
    // implemented logic  
}  
  
class MoreAwesomeArrayList implements List {  
    //implemented logic  
}
```

# KISS

Always Keep It Simple, Stupid  будь проще

- методы должны быть небольшими (40 - 50 строк)
- каждый метод решает одну проблему
- при модификации кода в будущем не должно возникнуть трудностей
- система работает лучше всего, если она не усложняется без надобности
- не устанавливайте целую библиотеку ради одной функции из неё
- не делай того, что не просят
- писать код необходимо надёжно и «дубово»

# YAGNI

You are not gonna need it - вам это не понадобится

- реализуйте только то, что нужно здесь и сейчас, а не в теории, что оно пригодится в будущем
- не добавляйте новый функционал, о котором его не просят (только добавьте багов)
- подчищайте ненужный код (найдете через Git историю при надобности)

# DRY

Don't Repeat Yourself – не повторяйся

- избегайте копирования кода
- выносите общую логику
- прежде чем добавлять функционал, проверьте в проекте, может, он уже создан
- константы

# Что такое Паттерн?

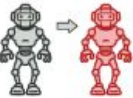
Паттерн проектирования — это часто встречающееся решение определённой проблемы при проектировании архитектуры программ.

Три основные группы GOF (GangOfFour) паттернов:

- Порождающие паттерны беспокоятся о гибком создании объектов без внесения в программу лишних зависимостей.
- Структурные паттерны показывают различные способы построения связей между объектами.
- Поведенческие паттерны заботятся об эффективной коммуникации между объектами.

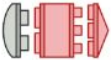
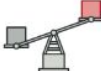
## Порождающие

Отвечают за удобное и безопасное создание новых объектов или даже целых семейств объектов.

|                                                                                                                               |                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <br><b>Фабричный метод</b><br>Factory Method | <br><b>Абстрактная фабрика</b><br>Abstract Factory |
| <br><b>Строитель</b><br>Builder              | <br><b>Прототип</b><br>Prototype                   |
| <br><b>Одиночка</b><br>Singleton             |                                                                                                                                     |

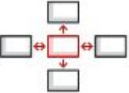
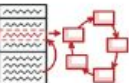
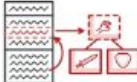
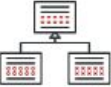
## Структурные

Отвечают за построение удобных в поддержке иерархий классов.

|                                                                                                                      |                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <br><b>Адаптер</b><br>Adapter       | <br><b>Мост</b><br>Bridge         |
| <br><b>Компоновщик</b><br>Composite | <br><b>Декоратор</b><br>Decorator |
| <br><b>Фасад</b><br>Facade          | <br><b>Легковес</b><br>Flyweight  |
| <br><b>Заместитель</b><br>Proxy     |                                                                                                                    |

## Поведенческие

Решают задачи эффективного и безопасного взаимодействия между объектами программы.

|                                                                                                                                               |                                                                                                                                  |                                                                                                                     |                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <br><b>Цепочка обязанностей</b><br>Chain of Responsibility | <br><b>Команда</b><br>Command                 | <br><b>Итератор</b><br>Iterator  | <br><b>Посредник</b><br>Mediator |
| <br><b>Снимок</b><br>Memento                               | <br><b>Наблюдатель</b><br>Observer            | <br><b>Состояние</b><br>State    |                                                                                                                     |
| <br><b>Стратегия</b><br>Strategy                           | <br><b>Шаблонный метод</b><br>Template Method | <br><b>Посетитель</b><br>Visitor |                                                                                                                     |

# GRASP

General Responsibility Assignment Software Patterns (паттерны распределения ответственности)

- 1) Information Expert
- 2) Creator
- 3) Controller
- 4) Low Coupling
- 5) High Cohesion
- 6) Pure Fabrication
- 7) Indirection
- 8) Protected Variations
- 9) Polymorphism

# Литература

- <https://habr.com/ru/post/508086/>
- [SOLID от Сергея Немчинского](#)
- <https://refactoring.guru/ru/design-patterns/catalog>